

Fintech Phase 1: Comprehensive System Documentation

1. Executive Summary

Fintech Phase 1 is a state-of-the-art, AI-powered financial technology platform designed to provide institutional-level analytics, data processing, and predictive modeling to retail investors. The system seamlessly integrates real-time market data with advanced machine learning models to enable data-driven investment decisions.

2. Global Architecture

The platform is designed as a scalable microservices architecture comprising three main compute nodes and two stateful data nodes:

- **Frontend (UI Node):** A Next.js (React) application serving as the user interface.
 - **Backend (Core API Node):** A FastAPI (Python) service handling business logic, user management, portfolio tracking, and trade execution.
 - **ML Service (Analytics Node):** A dedicated FastAPI (Python) service responsible for heavy computational workflows, model training, and inference.
 - **Database:** PostgreSQL extended with TimescaleDB for optimized time-series data storage.
 - **Cache & Message Broker:** Redis for session management, fast caching, and asynchronous task queueing.
-

3. Frontend Modules (Next.js Application)

The frontend is divided into several localized modules accessible via the internal dashboard router (/app/(dashboard)).

3.1 Dashboard (/dashboard)

Serves as the central command center for the user. It aggregates views from all other modules, displaying portfolio balance, daily P&L, top market movers, and active alerts.

3.2 Authentication (/auth)

Handles user onboarding, login, JWT token management, and session recovery.

3.3 Technical Analysis (/technical)

Provides advanced interactive charting powered by market data. Allows users to overlay indicators such as Moving Averages, RSI, MACD, and Bollinger Bands.

3.4 Fundamental Analysis (/fundamental)

Displays company health metrics, balance sheets, income statements, and cash flow information. It pairs with the ML service to present AI-summarized health scores.

3.5 ML Predictions (/risk & /strategies)

Visualizes the output of the ML models.

- **Risk Module:** Displays Value at Risk (VaR), portfolio beta, and Sharpe ratios.
- **Strategies Module:** Shows algorithmic trading strategies and their historical performance.

3.6 Backtesting Simulator (/backtest)

Provides an interactive form where users can define trading parameters (e.g., "Buy when 50-SMA crosses 200-SMA"), select a timeline, and view simulated historical returns against a benchmark.

3.7 Trading Module (/trade)

Facilitates paper trading (simulated execution). Users can place Market, Limit, and Stop orders. Connects directly to the backend Order module.

3.8 Community & News (/community, /news)

- **Community:** A social feed where users can share trades, analyses, and discuss market trends.
- **News:** Real-time financial news aggregation, filtered by the user's portfolio assets and enhanced with NLP sentiment scores.

3.9 Education (/learn)

A structured learning management system within the app, offering modules on financial terminology, fundamental analysis, technical analysis, and risk management.

4. Backend Service (Core API)

The Core API handles standard CRUD operations and state management.

4.1 Data Models (backend/app/models)

- **User:** Manages authentication details, tiered roles, and preferences.
- **Stock:** Stores master data for available equities, tickers, and standard metadata.
- **Portfolio & Order:** Tracks simulated user balances, active positions, and order history (execution logs).
- **Alert:** Stores user-configured conditions (e.g., "Alert when AAPL drops 5%").
- **Education, News, Social:** Manages courses, aggregated articles, and user forum posts.
- **Strategy:** Tracks user-defined logic rules for custom automated trading configurations.

4.2 API Endpoints (backend/app/api/v1/endpoints)

- **/auth:** JWT generation, token refresh, and user registration.
 - **/users:** Profile management and setting modifications.
 - **/stocks:** Fetching ticker lists, standard historical data, and company metadata.
 - **/trading:** Submitting, modifying, and canceling paper trade orders.
 - **/alerts:** CRUD operations for user price & technical alerts.
 - **/strategies:** Endpoints to save user-defined algorithmic rules.
 - **/social & /education:** Fetch feeds, post replies, and track course completion progress.
-

5. Machine Learning Service (ML Analytics)

This dedicated service offloads heavy computations blocking the main API. It exposes endpoints consumed by the Core Backend or directly by the Frontend.

5.1 Training Service (training_service.py)

Responsible for retraining predictive models (e.g., LSTM/Transformer) on a scheduled basis using updated historical data.

5.2 Prediction Service (prediction_service.py)

Provides inference. Given a stock ticker, it outputs a forecast for the next N days along with a confidence interval.

5.3 Risk Service (risk_service.py)

Calculates complex combinatorial portfolio risk. Evaluates maximum drawdown scenarios, Monte Carlo simulations for portfolio trajectories, and standard metrics (Alpha, Beta).

5.4 Backtest Service (backtest_service.py)

A simulation engine that replays historical market data against user-defined logic rules, calculating simulated P&L, win rates, and maximum drawdowns.

5.5 Sentiment Service (sentiment_service.py)

Applies NLP (Natural Language Processing) to daily news headlines and social media posts (e.g., Twitter feeds) to assign a bullish or bearish score to a specific ticker.

5.6 Feature & Fundamental Service (feature_service.py, fundamental_service.py)

Handles data preprocessing. Cleans raw historical price data, calculates advanced rolling technical indicators (features for ML), and parsing raw fundamental data into normalized ratios.

6. Infrastructure & Deployment

- **Docker Compose:** The entire suite spans 5 containers (frontend, backend, ml-service, db, redis), networked internally to ensure isolation and security.
- **Database (TimescaleDB):** Critical for efficiently querying thousands of rows of candlestick price data per second.
- **Redis:** Operates as both a high-speed cache for frequently requested data (like current prices) and a broker for background tasks (e.g., triggering a model retraining cycle).

7. Future Scaling (Startup Roadmap)

- **Real-Time Data Streams:** Implementation of WebSocket channels to beam sub-millisecond price updates.
- **Live Broker Integration:** Hooking up the trade module to live clearinghouses (e.g., Alpaca, Interactive Brokers) to shift from paper trading to real capital execution.
- **Kubernetes (K8s) Orchestration:** Container auto-scaling, specifically allowing the ml-service to

spawn replicated nodes during computationally heavy market hours.